STUDENT BUDGET AND EXPENSE TRACKER

Software Engineering Project Documentation

Prepared by: Project Group of 10 Students

PHASE ONE: PROJECT INTRODUCTION

1.1 Project Title

Student Budget and Expense Tracker.

1.2 Background of the Project

Many university students receive money from parents, guardians, scholarships, businesses or personal sources. However, students often struggle to manage their finances effectively.

Some students spend money without recording their transactions, forget how much they have spent, exceed their available budgets and find it difficult to determine where their money goes.

Some rely on notebooks, calculators, memory or simple spreadsheets to track their finances. These approaches can be inconvenient and may not provide automatic calculations, organized reports or timely budget warnings.

The proposed Student Budget and Expense Tracker will provide a digital platform that allows students to record their income and expenses, organize transactions, establish budgets and monitor their financial activities.

1.3 Statement of the Problem

The major problems this project intends to address include:

1. Poor tracking of daily income and expenses.
2. Difficulty determining the actual available balance.
3. Overspending due to the absence of proper budget monitoring.
4. Difficulty identifying categories where money is spent most.

5. Lack of organized financial records.
6. Manual calculations that may result in errors.
7. Difficulty generating financial summaries and reports.

1.4 Aim of the Project

To design and develop a Student Budget and Expense Tracker that helps university students record, organize, monitor and analyze their personal finances.

1.5 Objectives of the Project

The system should:

1. Allow students to register and log in securely.
2. Allow users to record their income.
3. Allow users to record their expenses.
4. Organize expenses into categories.
5. Allow users to create daily, weekly and monthly budgets.
6. Automatically calculate total income, total expenses and available balance.
7. Monitor spending against established budgets.
8. Notify users when they approach or exceed their budget limits.
9. Display financial information through an interactive dashboard.
10. Generate financial reports for selected periods.
11. Allow users to view, edit, delete and filter transactions.
12. Protect users' financial records from unauthorized access.

1.6 Scope of the Project

Target users. The primary users are university students who want to manage their personal finances. The initial version is designed with Nigerian university students in mind.

Currency. Nigerian Naira (₦) is the default and only supported currency in the initial version. Monetary values are formatted properly, for example ₦5,000.00.

Features included in the initial version:

- User registration.
- Login and logout.

- User profile management.
- Income recording and management.
- Expense recording and management.
- Expense categories.
- Budget creation and management.
- Daily, weekly and monthly budget periods.
- Automatic balance calculations.
- Financial dashboard.
- Transaction history.
- Transaction search and filtering.
- Budget monitoring.
- Budget warning notifications.
- Financial reports.
- Responsive interface for mobile phones, tablets and computers.

Features excluded from the initial version:

- Bank account integration.
- Automatic bank transaction retrieval.
- Money transfers between users.
- Loan management.
- Investment management.
- Cryptocurrency management.
- Payment gateway integration.

The system relies on users manually entering their income and expenses.

1.7 Significance of the Project

The project will help students to keep accurate records of their money, understand their spending habits and avoid overspending. It also gives the project group practical experience in applying the software development life cycle (SDLC) to a real problem, from requirements analysis through design, implementation, testing and deployment.

1.8 Platform Approach

The group wants both a web application and an Android-accessible application, but has limited development time and resources. The project therefore uses a Progressive Web Application (PWA) approach. The application works in web browsers, has a responsive mobile interface, can be installed on the home screen of compatible Android devices and gives an app-like experience, all from one shared codebase.

A PWA is not a native Android application. If a lecturer specifically requires a native Android APK, additional development would be necessary, for example wrapping the PWA in an Android container or building a separate native application.

1.9 Development Methodology

The project follows the software development life cycle in this order: requirements analysis, feasibility study, system design, user interface design, implementation, testing, deployment and maintenance. The team works iteratively, so that each feature (for example income recording) is designed, built and tested before the next one begins. Git and GitHub are used for version control.

PHASE TWO: REQUIREMENTS ANALYSIS

2.1 Introduction

Requirements analysis is the process of identifying, examining and documenting the needs and expectations of users regarding a proposed software system.

For the Student Budget and Expense Tracker, this phase focuses on identifying the functions the system must perform, the conditions under which it must operate and the expectations of its users.

The outcome of this phase will serve as a foundation for the system design and implementation stages.

2.2 Identification of Stakeholders

Stakeholders are individuals or groups who interact with, benefit from or have an interest in the proposed system.

The major stakeholders include:

1. Students (Primary Users)

Students are the main users of the application. They will:

- Register and log into the system.
- Record their income.
- Record their expenses.
- Create and manage budgets.
- Monitor their available balances.
- View financial reports.

2. Project Administrators

Administrators are responsible for managing the application's technical environment and ensuring its proper operation.

Their responsibilities may include:

- Monitoring system performance.
- Managing application settings.
- Handling technical issues.
- Maintaining the application.

Note: Administrative access is a proposed supporting role, not a requirement for the initial student-facing MVP.

3. Project Development Team

The development team is responsible for analysing, designing, implementing, testing and maintaining the application.

2.3 User Requirements

User requirements describe what users expect from the application in simple terms.

The system should allow students to:

1. Create personal accounts.
2. Log into their accounts securely.
3. Record money received from different sources.
4. Record their daily expenditures.
5. Organize transactions into categories.
6. Create daily, weekly and monthly budgets.
7. View their total income and expenses.

8. Check their remaining financial balance.
9. Receive alerts when approaching or exceeding budget limits.
10. View their previous transactions.
11. Generate financial summaries and reports.
12. Edit or delete incorrect transaction records.

2.4 Functional Requirements

Functional requirements describe the specific operations and services the system must provide.

The following functional requirements have been identified.

FR01: User Registration

The system shall allow new students to create accounts by providing their names, email addresses and passwords.

FR02: User Authentication

The system shall allow registered students to log in using their registered email addresses and passwords.

FR03: User Logout

The system shall allow authenticated users to securely log out of their accounts.

FR04: User Profile Management

The system shall allow users to view and update their personal profile information.

FR05: Add Income

The system shall allow users to record income by entering the amount, source, date and description.

Examples of income sources include allowances, scholarships, part-time employment and business earnings.

FR06: Manage Income Records

The system shall allow users to view, edit and delete previously recorded income transactions.

FR07: Record Expenses

The system shall allow users to record expenses by entering the amount, category, date and description.

FR08: Categorize Expenses

The system shall allow expenses to be organized into categories such as:

- Feeding
- Transportation
- Accommodation
- Academic materials
- Internet and data
- Healthcare
- Entertainment
- Other expenses

FR09: Manage Expense Records

The system shall allow users to view, edit and delete previously recorded expenses.

FR10: Create Budgets

The system shall allow users to establish spending limits for selected expense categories over daily, weekly or monthly periods.

FR11: Monitor Budgets

The system shall automatically compare recorded expenses with their corresponding budget limits.

FR12: Calculate Financial Balance

The system shall automatically calculate the user's available balance using total recorded income minus total recorded expenses.

FR13: Financial Dashboard

The system shall provide a dashboard displaying:

- Total income.
- Total expenses.
- Available balance.
- Budget utilization.
- Recent transactions.

- Spending summaries.

FR14: Generate Financial Reports

The system shall generate summaries of users' income and expenditure over selected periods.

FR15: Budget Notifications

The system shall notify users when their spending approaches or exceeds an established budget limit.

FR16: Transaction History

The system shall maintain a history of recorded income and expense transactions and allow users to retrieve their records.

FR17: Transaction Filtering

The system shall allow users to filter transactions by date, category and transaction type.

2.5 Non-Functional Requirements

Non-functional requirements describe the quality attributes and operational conditions the system must satisfy.

NFR01: Usability

The system shall provide a simple, understandable and user-friendly interface that students can navigate easily.

NFR02: Performance

The system shall process normal user operations and display financial summaries within a reasonable response time.

NFR03: Security

The system shall protect user accounts and financial records through secure authentication, password hashing and appropriate access controls.

NFR04: Reliability

The system shall perform financial calculations consistently and maintain accurate transaction records.

NFR05: Availability

The system should be accessible whenever the hosting service and internet connection are available.

NFR06: Compatibility

The system shall support modern web browsers and compatible Android devices.

NFR07: Responsiveness

The interface shall adjust appropriately to different screen sizes, including mobile phones, tablets and laptops.

NFR08: Maintainability

The system shall be structured in a way that allows developers to modify, debug and improve its components.

NFR09: Data Integrity

The system shall validate financial entries and prevent invalid or incomplete transaction records from being stored.

NFR10: Scalability

The system should be designed to accommodate additional users and features as its usage increases.

2.6 System Constraints

The following constraints have been identified:

1. The initial version will primarily support Nigerian naira.
2. Users will manually enter their income and expenses.
3. Access to the hosted application will require an internet connection.
4. The application must be completed within the available project deadline.
5. Development will depend on the technical skills and resources available to the group.
6. The initial application will not directly integrate with banking institutions.

2.7 Business Rules

Business rules define the conditions that govern how the application operates.

BR01: Account Ownership

Each student must have an individual account, and financial records must belong to the account that created them.

BR02: Positive Financial Entries

Income, expense and budget amounts must be greater than zero.

BR03: Balance Calculation

Available balance must be calculated as:

Available Balance = Total Income − Total Expenses.

BR04: Budget Monitoring

The system must compare the expenses assigned to a budget category with the established spending limit. Budget Utilization (%) = (Total Expenses in Budget Category ÷ Budget Limit) × 100.

BR05: Budget Warning

The system should notify a student when spending reaches 80% of a budget limit and when the limit is exceeded. The 80% threshold is configurable in the code so that it can be adjusted later.

BR06: Transaction Ownership

Users must only be able to access and modify their own financial transactions.

BR07: Transaction Editing

When a transaction is edited or deleted, the dashboard and financial summaries must reflect the updated records.

BR08: Currency

All financial transactions in the initial version shall be recorded in Nigerian naira.

2.8 System Actors

The actors identified for the initial system are:

Actor	Responsibility
Student	Records income, expenses, manages budgets and views reports
Administrator	Handles application-level technical management

The Student is the primary actor because most application activities are initiated by the student.

2.9 Major System Use Cases

The major use cases identified include:

1. Register account.
2. Log in.
3. Log out.
4. Manage profile.
5. Add income.
6. View income records.
7. Edit income.
8. Delete income.
9. Record expense.
10. Categorize expense.
11. View expense records.
12. Edit expense.
13. Delete expense.
14. Create budget.
15. Monitor budget.
16. View financial dashboard.
17. Generate financial report.
18. View transaction history.
19. Filter transactions.
20. Receive budget notifications.

2.10 Input, Processing and Output Requirements

A. Input Requirements

The system will accept the following information:

- User registration details.
- Login credentials.
- Income amounts and sources.

- Expense amounts and categories.
- Transaction dates and descriptions.
- Budget amounts and periods.

B. Processing Requirements

The system will:

- Validate user inputs.
- Authenticate registered users.
- Store financial transactions.
- Calculate total income.
- Calculate total expenses.
- Determine available balances.
- Compare expenses with budget limits.
- Generate financial summaries.

C. Output Requirements

The system will display:

- Registration and login confirmations.
- Income and expense records.
- Available financial balances.
- Budget utilization information.
- Budget warning notifications.
- Spending charts and financial reports.
- Transaction histories.

2.11 Requirement Validation

The identified requirements should be reviewed and validated by the project team and potential student users before implementation.

Validation will help establish that:

- The requirements address the actual financial management problems faced by students.
- All essential system functions have been identified.
- The requirements are understandable and achievable.

- The requirements are consistent with the project objectives.
- The proposed features can be implemented within the available time and resources.

2.12 Conclusion

The requirements analysis phase has identified the major users, expected system functionalities, quality requirements, business rules and operational constraints of the Student Budget and Expense Tracker.

These requirements will guide the next stage of development, which involves translating the identified requirements into a detailed system design.

PHASE THREE: FEASIBILITY STUDY

3.1 Introduction

A feasibility study examines whether the proposed system can be developed successfully with the technology, money, skills and time that are available. It helps the project team to avoid starting a project that cannot be completed.

3.2 Technical Feasibility

The project uses accessible technologies: Python, the Flask framework, SQLite, HTML5, CSS3 and JavaScript. These technologies are free, well documented and widely taught, and the group members can learn them within the project period. A Progressive Web Application (PWA) approach allows one shared codebase to serve both web browsers and compatible Android devices, which avoids building two separate applications.

The project is therefore considered technically feasible.

3.3 Economic Feasibility

The initial development can be completed using free or low-cost tools and open-source technologies. Python, Flask, SQLite, Git, GitHub, VS Code and Figma (free plan) do not require paid licences for a student project. Hosting can be done on a free or low-cost platform such as Render, but its current limits and pricing must be checked before deployment, because free hosting plans can change and may limit performance or storage.

The project is therefore considered economically feasible.

3.4 Operational Feasibility

The application is designed to be easy for ordinary university students to understand and operate. It uses a simple interface, familiar forms, clear categories and an automatic dashboard, so students do not need technical training. Students already track money using notebooks or spreadsheets, so the proposed system fits into an activity they already perform.

The project is therefore considered operationally feasible.

3.5 Schedule Feasibility

The system must be developed within the available academic project period. To make this possible, the initial version is limited to the features in the project scope, and complex features such as bank integration, payment gateways and native Android development are excluded. Work is divided among the 10 group members so that design, development, testing and documentation can proceed in parallel.

The project is therefore considered schedule feasible, provided the scope is controlled.

3.6 Risks and Mitigation

Risk	Effect	Mitigation
Limited development time	Features may not be completed	Build the core features first and delay optional features
Limited technical skill	Slower development	Use simple technologies and share knowledge among members
Free hosting limitations	Slow or unavailable application	Check hosting limits before deployment and keep a local copy that can run
Calculation errors	Wrong balances and budget alerts	Store money as exact values and write unit tests for calculations
Security weaknesses	Exposure of financial records	Apply password hashing, authorization checks and backend validation
Team coordination problems	Conflicting code	Use Git branches and pull requests on GitHub

3.7 Conclusion

The study shows that the Student Budget and Expense Tracker is technically, economically, operationally and schedule feasible. The project should proceed to system design.

PHASE FOUR: SYSTEM DESIGN

4.1 Introduction

System design translates the requirements from Phase Two into models that developers can follow when building the application. This phase uses standard UML notation for the use case, activity and sequence diagrams, followed by the system architecture and the database design.

4.2 Use Case Model

A use case diagram shows what the actors can do with the system. The system boundary is the Student Budget and Expense Tracker.

Actors

- **Student:** the primary actor.
- **Administrator:** a supporting actor with the single use case Manage Application.

Use cases of the Student

Register, Login, Logout, Manage Profile, Manage Income, Manage Expenses, Manage Categories, Create Budget, Monitor Budget, View Dashboard, View Transaction History, Filter Transactions, Generate Financial Reports and Receive Budget Notifications.

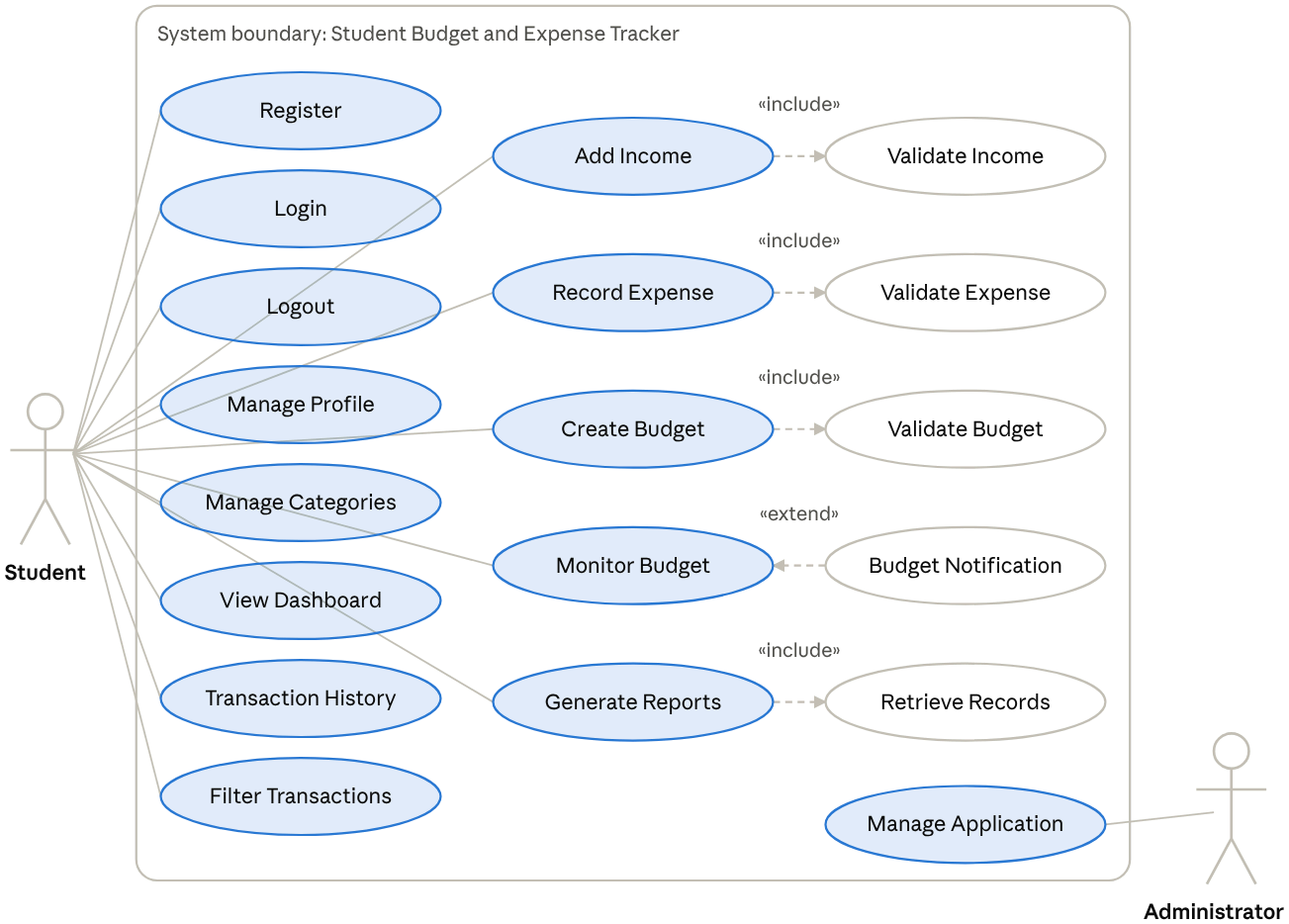
Use case relationships

Relationship	Base use case	Included or extending use case	Meaning
«include»	Record Expense	Validate Expense Details	Every expense is validated before it is saved
«include»	Add Income	Validate Income Details	Every income record is validated before it is saved

Relationship	Base use case	Included or extending use case	Meaning
«include»	Create Budget	Validate Budget Details	Every budget is validated before it is saved
«include»	Generate Financial Report	Retrieve Financial Records	A report always retrieves the user's records
«extend»	Monitor Budget	Receive Budget Notification	The notification occurs only when a threshold is reached (80% or 100%)

In UML notation, an include arrow is a dashed arrow labelled «include» that points from the base use case to the included use case. An extend arrow is a dashed arrow labelled «extend» that points from the extending use case to the base use case.

The Student uses 14 use cases; the Administrator manages the application



UML use case diagram · 2 actors, 19 use cases

Figure 4.1: Use case diagram. Add Income and Record Expense are the recording parts of Manage Income and Manage Expenses. Dashed arrows show the include and extend relationships described above.

4.3 Activity Diagrams

Activity diagrams show the flow of actions, decisions and alternative paths. In the diagrams below, the circle labelled Start is the initial node, rectangles are actions, diamonds are decision points and the circle labelled End is the final node.

4.3.1 User Registration and Login

```
flowchart TD
  A((Start)) --> B[Open registration page]
  B --> C[Enter name, email and password]
  C --> D[Submit form]
  D --> E{Details valid?}
  E -- No --> F[Show validation errors] --> C
  E -- Yes --> G{Email already registered?}
  G -- Yes --> H[Show email already exists message] --> C
  G -- No --> I[Hash password and save user]
  I --> J[Open login page]
  J --> K[Enter email and password]
  K --> L{Credentials correct?}
  L -- No --> M[Show login error] --> K
  L -- Yes --> N[Create secure session]
  N --> O[Open dashboard]
  O --> P((End))
```

4.3.2 Recording an Expense

```
flowchart TD
  A((Start)) --> B[Open Add Expense form]
  B --> C[Enter amount, category, date and description]
  C --> D[Tap Save]
  D --> E{Session valid?}
  E -- No --> F[Redirect to login] --> Z((End))
  E -- Yes --> G{Amount above zero and category selected?}
  G -- No --> H[Show error message] --> C
  G -- Yes --> I[Save expense linked to the user]
  I --> J{Database save successful?}
  J -- No --> K[Show system error] --> Z
  J -- Yes --> L[Recalculate totals and budget utilization]
  L --> M{Utilization at 80 percent or more?}
  M -- Yes --> N[Show warning or over-budget alert]
  N --> O[Show success message]
  M -- No --> O
  O --> Z
```

4.3.3 Adding Income

```
flowchart TD
  A((Start)) --> B[Open Add Income form]
  B --> C[Enter amount, source, date and description]
  C --> D[Tap Save]
  D --> E{Session valid?}
  E -- No --> F[Redirect to login] --> Z((End))
  E -- Yes --> G{Amount above zero and source entered?}
  G -- No --> H[Show error message] --> C
  G -- Yes --> I[Save income linked to the user]
  I --> J{Database save successful?}
  J -- No --> K[Show system error] --> Z
  J -- Yes --> L[Recalculate total income and available balance]
  L --> M[Show success message and updated balance]
  M --> Z
```

4.3.4 Creating a Budget

```
flowchart TD
  A((Start)) --> B[Open Create Budget form]
  B --> C[Select category]
  C --> D[Enter budget amount]
  D --> E[Select daily, weekly or monthly period]
  E --> F[Select start date]
  F --> G[System calculates end date]
  G --> H{Amount above zero, category and period valid?}
  H -- No --> I[Show error message] --> C
  H -- Yes --> J{Active budget already exists for this category and period?}
  J -- Yes --> K[Show duplicate budget message] --> C
  J -- No --> L[Save budget linked to the user]
  L --> M[Show budget created confirmation]
  M --> N((End))
```

4.3.5 Monitoring a Budget

```
flowchart TD
  A((Start)) --> B[Expense is added, edited or deleted, or budget page is
opened]
  B --> C[Find the user's budgets covering the expense date]
  C --> D{Any matching budget?}
  D -- No --> Z((End))
  D -- Yes --> E[Add up expenses in the budget category and period]
  E --> F[Calculate utilization = expenses / limit x 100]
  F --> G{Utilization level}
  G -- Below 80 percent --> H[Show normal progress]
  G -- 80 to 100 percent --> I{Warning already shown for this status?}
  G -- Above 100 percent --> J{Over-budget alert already shown for this
status?}
  I -- No --> K[Show warning notification and record status]
  I -- Yes --> H
  J -- No --> L[Show over-budget alert and record status]
  J -- Yes --> H
  H --> Z
  K --> Z
  L --> Z
```

4.3.6 Generating Financial Reports

```
flowchart TD
  A((Start)) --> B[Open Reports page]
  B --> C[Select start date and end date]
  C --> D{Start date on or before end date?}
  D -- No --> E[Show date range error] --> C
  D -- Yes --> F[Retrieve the user's income and expense records for the
period]
  F --> G{Any records found?}
  G -- No --> H[Show empty report message] --> Z((End))
  G -- Yes --> I[Calculate total income, total expenses and balance]
  I --> J[Calculate spending by category and budget performance]
  J --> K[Display report]
  K --> L{Print or download?}
  L -- Yes --> M[Produce printable report]
  M --> Z
  L -- No --> Z
```

4.4 Sequence Diagrams

Sequence diagrams show the messages exchanged between the Student, the Frontend Interface, the Flask Backend and the SQLite Database over time.

4.4.1 User Login

```
sequenceDiagram
  actor S as Student
  participant F as Frontend Interface
  participant B as Flask Backend
  participant D as SQLite Database
  S->>F: Enter email and password
  F->>B: POST /login with CSRF token
  B->>B: Validate input format
  B->>D: SELECT user WHERE email = ?
  D-->>B: User record or none
  alt User found and password hash matches
    B->>B: Create secure session
    B-->>F: Redirect to dashboard
    F-->>S: Display dashboard
  else Invalid credentials
    B-->>F: Return generic login error
    F-->>S: Show Invalid email or password
  end
```

4.4.2 Recording an Expense

```
sequenceDiagram
  actor S as Student
  participant F as Frontend Interface
  participant B as Flask Backend
  participant D as SQLite Database
  S->>F: Fill expense form and tap Save
  F->>B: POST /expenses with CSRF token
  B->>B: Check session and validate amount, category and date
  alt Validation fails
    B-->>F: 400 error with message
    F-->>S: Show error, nothing saved
  else Validation passes
    B->>D: INSERT expense with user_id
    D-->>B: Saved
    B->>D: Query totals and budget utilization
    D-->>B: Results
    B-->>F: 201 response with totals and alert status
    F-->>S: Show success and any warning
  end
```

4.4.3 Adding Income

```
sequenceDiagram
  actor S as Student
  participant F as Frontend Interface
  participant B as Flask Backend
  participant D as SQLite Database
  S->>F: Fill income form and tap Save
  F->>B: POST /income with CSRF token
  B->>B: Check session and validate amount, source and date
  alt Validation fails
    B-->>F: 400 error with message
    F-->>S: Show error, nothing saved
  else Validation passes
    B->>D: INSERT income with user_id
    D-->>B: Saved
    B->>D: Query total income and total expenses
    D-->>B: Totals
    B-->>F: 201 response with new balance
    F-->>S: Show success and updated balance
  end
```

4.4.4 Creating a Budget

```
sequenceDiagram
  actor S as Student
  participant F as Frontend Interface
  participant B as Flask Backend
  participant D as SQLite Database
  S->>F: Enter category, amount, period and start date
  F->>B: POST /budgets with CSRF token
  B->>B: Check session, validate data and calculate end date
  alt Validation fails
    B-->>F: 400 error with message
    F-->>S: Show error
  else Validation passes
    B->>D: SELECT existing active budget for category and period
    D-->>B: Result
    alt Duplicate exists
      B-->>F: 409 duplicate budget message
      F-->>S: Show duplicate message
    else No duplicate
      B->>D: INSERT budget with user_id
      D-->>B: Saved
```

```
      B-->>F: 201 response
      F-->>S: Show budget created
    end
  end
```

4.4.5 Generating Financial Reports

```
sequenceDiagram
  actor S as Student
  participant F as Frontend Interface
  participant B as Flask Backend
  participant D as SQLite Database
  S->>F: Select report period and tap Generate
  F->>B: GET /reports with start and end dates
  B->>B: Check session and validate date range
  alt Invalid date range
    B-->>F: 400 error with message
    F-->>S: Show date range error
  else Valid date range
    B->>D: SELECT income, expenses and budgets WHERE user_id = ? AND date in
range
    D-->>B: Records
    B->>B: Calculate totals, category spending and budget performance
    B-->>F: Report data
    F-->>S: Display report with print option
  end
```

4.5 System Architecture

The system follows a client-server architecture. The student uses the Progressive Web Application in a browser or from the Android home screen. The frontend sends requests to the Flask backend, and the backend reads and writes data in the SQLite database.

```
flowchart LR
  S["Student"] --> F["Frontend PWA: HTML, CSS, JavaScript, manifest, service
worker"]
  F -->|"HTTPS requests"| B["Flask Backend: routes, authentication,
validation, business logic"]
  B -->|"SQL queries"| D[("SQLite Database")]
  D -->|"Query results"| B
  B -->|"HTML or JSON responses"| F
  F --> S
```

Component	Responsibility
Student	Uses a phone, tablet or computer to enter income, expenses and budgets and to read the dashboard and reports
Frontend (PWA)	Displays pages, forms, charts and notifications; performs basic input checks for convenience; provides the web app manifest and service worker that make the application installable
Flask Backend	Handles routes, user sessions, authentication, authorization, CSRF protection, server-side validation, balance and budget calculations, and all database access
SQLite Database	Stores users, categories, income, expenses, budgets and budget alerts permanently, with constraints that reject invalid data

The frontend is never trusted to validate financial information on its own. The backend validates every important value again before it is stored.

4.6 Database Design

4.6.1 Design decisions

- **One database, many users.** Every record that belongs to a student carries that student's user_id, and every query filters by the logged-in user's user_id. Students therefore cannot see or change each other's records.
- **Exact money values.** Floating-point numbers can produce inaccurate results such as 0.1 + 0.2 not equalling 0.3. Amounts are stored as whole numbers of kobo (₦1 = 100 kobo) in INTEGER columns and converted to naira only for display, for example 500000 kobo is shown as ₦5,000.00. This is a proposed improvement over storing decimal amounts.
- **Default and custom categories.** A category with a NULL user_id is a default category shared by everyone. A category with a user_id is a custom category belonging to that student.

- **Avoiding duplicate alerts.** An extra table, budget_alerts, records which alert status has already been shown for a budget so that the same alert is not repeated unnecessarily.
- **Column names.** The date columns are named income_date and expense_date instead of date to avoid confusion with SQL keywords.

4.6.2 Entity relationship diagram

```
erDiagram
  USERS ||--o{ CATEGORIES : creates
  USERS ||--o{ INCOME : records
  USERS ||--o{ EXPENSES : records
  USERS ||--o{ BUDGETS : sets
  CATEGORIES ||--o{ EXPENSES : classifies
  CATEGORIES ||--o{ BUDGETS : limits
  BUDGETS ||--o{ BUDGET_ALERTS : triggers
  USERS ||--o{ BUDGET_ALERTS : receives
  USERS {
    INTEGER user_id PK
    TEXT full_name
    TEXT email UK
    TEXT password_hash
    TEXT created_at
  }
  CATEGORIES {
    INTEGER category_id PK
    INTEGER user_id FK
    TEXT category_name
    TEXT category_type
  }
  INCOME {
    INTEGER income_id PK
    INTEGER user_id FK
    INTEGER amount_kobo
    TEXT source
    TEXT description
    TEXT income_date
    TEXT created_at
  }
  EXPENSES {
    INTEGER expense_id PK
    INTEGER user_id FK
    INTEGER category_id FK
```

```
    INTEGER amount_kobo
    TEXT description
    TEXT expense_date
    TEXT created_at
  }
  BUDGETS {
    INTEGER budget_id PK
    INTEGER user_id FK
    INTEGER category_id FK
    INTEGER budget_amount_kobo
    TEXT budget_period
    TEXT start_date
    TEXT end_date
    TEXT created_at
  }
  BUDGET_ALERTS {
    INTEGER alert_id PK
    INTEGER user_id FK
    INTEGER budget_id FK
    TEXT alert_type
    TEXT created_at
  }
```

4.6.3 Relationships

Relationship	Type	Meaning
Users to Categories	One to many (optional)	A student may create many custom categories; default categories have no user
Users to Income	One to many	A student records many income entries
Users to Expenses	One to many	A student records many expenses
Users to Budgets	One to many	A student sets many budgets
Categories to Expenses	One to many	Every expense belongs to exactly one category
Categories to Budgets	One to many	Every budget limits exactly one category
Budgets to Budget_Alerts	One to many	A budget can trigger a warning alert and an over-budget alert

4.6.4 Data dictionary

Users

Column	Data type	Key and constraints	Description
user_id	INTEGER	Primary key, auto-increment	Unique identifier of the student
full_name	TEXT	Not null	Student's full name
email	TEXT	Not null, unique	Login email address
password_hash	TEXT	Not null	Hashed password; the plain password is never stored
created_at	TEXT (timestamp)	Not null, default current time	Date the account was created

Categories

Column	Data type	Key and constraints	Description
category_id	INTEGER	Primary key	Unique identifier of the category
user_id	INTEGER	Foreign key to Users, may be null	Owner of a custom category; null for default categories
category_name	TEXT	Not null	Name such as Feeding or Transportation
category_type	TEXT	Not null, expense or income	Kind of category

Income

Column	Data type	Key and constraints	Description
income_id	INTEGER	Primary key	Unique identifier of the income record
user_id	INTEGER	Foreign key to Users, not null	Owner of the record
amount_kobo	INTEGER	Not null, greater than zero	Amount in kobo
source	TEXT	Not null	Allowance, scholarship, part-time employment, business earnings, etc.
description	TEXT	Optional	Extra notes
income_date	TEXT (date)	Not null	Date the money was received
created_at	TEXT (timestamp)	Not null	Date the record was created

Expenses

Column	Data type	Key and constraints	Description
expense_id	INTEGER	Primary key	Unique identifier of the expense
user_id	INTEGER	Foreign key to Users, not null	Owner of the record
category_id	INTEGER	Foreign key to Categories, not null	Category of the expense
amount_kobo	INTEGER	Not null, greater than zero	Amount in kobo
description	TEXT	Optional	Extra notes
expense_date	TEXT (date)	Not null	Date the money was spent
created_at	TEXT (timestamp)	Not null	Date the record was created

Budgets

Column	Data type	Key and constraints	Description
budget_id	INTEGER	Primary key	Unique identifier of the budget
user_id	INTEGER	Foreign key to Users, not null	Owner of the budget
category_id	INTEGER	Foreign key to Categories, not null	Category being limited
budget_amount_kobo	INTEGER	Not null, greater than zero	Spending limit in kobo
budget_period	TEXT	Not null, daily, weekly or monthly	Length of the budget
start_date	TEXT (date)	Not null	First day covered
end_date	TEXT (date)	Not null, on or after start_date	Last day covered

Column	Data type	Key and constraints	Description
created_at	TEXT (timestamp)	Not null	Date the budget was created

Budget_Alerts

Column	Data type	Key and constraints	Description
alert_id	INTEGER	Primary key	Unique identifier of the alert
user_id	INTEGER	Foreign key to Users, not null	Student who receives the alert
budget_id	INTEGER	Foreign key to Budgets, not null	Budget that triggered the alert
alert_type	TEXT	Not null, warning or over_budget	Alert status already shown
created_at	TEXT (timestamp)	Not null	Date the alert was created

The pair (budget_id, alert_type) is unique, so one alert of each type is recorded per budget. If editing or deleting transactions brings spending back below a threshold, the application should remove the matching alert record so that the alert can appear again if the threshold is crossed later.

4.6.5 Database creation script (SQLite)

```
PRAGMA foreign_keys = ON;

CREATE TABLE users (
  user_id       INTEGER PRIMARY KEY AUTOINCREMENT,
  full_name     TEXT NOT NULL,
  email         TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at    TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE categories (
  category_id   INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  user_id       INTEGER REFERENCES users(user_id) ON DELETE CASCADE,
  category_name TEXT NOT NULL,
  category_type TEXT NOT NULL DEFAULT 'expense'
                CHECK (category_type IN ('expense', 'income')),
  UNIQUE (user_id, category_name, category_type)
);

CREATE TABLE income (
  income_id    INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id      INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  amount_kobo  INTEGER NOT NULL CHECK (amount_kobo > 0),
  source       TEXT NOT NULL,
  description  TEXT,
  income_date  TEXT NOT NULL,
  created_at   TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE expenses (
  expense_id   INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id      INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  category_id  INTEGER NOT NULL REFERENCES categories(category_id) ON DELETE
RESTRICT,
  amount_kobo  INTEGER NOT NULL CHECK (amount_kobo > 0),
  description  TEXT,
  expense_date TEXT NOT NULL,
  created_at   TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE budgets (
  budget_id          INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id            INTEGER NOT NULL REFERENCES users(user_id) ON DELETE
CASCADE,
  category_id        INTEGER NOT NULL REFERENCES categories(category_id) ON
DELETE RESTRICT,
  budget_amount_kobo INTEGER NOT NULL CHECK (budget_amount_kobo > 0),
  budget_period      TEXT NOT NULL CHECK (budget_period IN ('daily',
'weekly', 'monthly')),
  start_date         TEXT NOT NULL,
  end_date           TEXT NOT NULL,
  created_at         TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK (end_date >= start_date)
);
```

```
CREATE TABLE budget_alerts (
  alert_id   INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id    INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  budget_id  INTEGER NOT NULL REFERENCES budgets(budget_id) ON DELETE
CASCADE,
  alert_type TEXT NOT NULL CHECK (alert_type IN ('warning', 'over_budget')),
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (budget_id, alert_type)
);

CREATE INDEX idx_income_user_date   ON income (user_id, income_date);
CREATE INDEX idx_expenses_user_date ON expenses (user_id, expense_date);
CREATE INDEX idx_expenses_category  ON expenses (category_id);
CREATE INDEX idx_budgets_user       ON budgets (user_id, start_date,
end_date);

-- Default categories shared by all users
INSERT INTO categories (user_id, category_name, category_type) VALUES
  (NULL, 'Feeding', 'expense'),
  (NULL, 'Transportation', 'expense'),
  (NULL, 'Accommodation', 'expense'),
  (NULL, 'Academic materials', 'expense'),
  (NULL, 'Internet and data', 'expense'),
  (NULL, 'Healthcare', 'expense'),
  (NULL, 'Entertainment', 'expense'),
  (NULL, 'Personal care', 'expense'),
  (NULL, 'Other expenses', 'expense');
```

Note that SQLite does not enforce foreign keys unless PRAGMA foreign_keys = ON is executed for each database connection, so the Flask application must run it every time it opens a connection.

4.6.6 Key calculations

- Total Income = sum of amount_kobo in the Income table for the logged-in user.
- Total Expenses = sum of amount_kobo in the Expenses table for the logged-in user.
- Available Balance = Total Income − Total Expenses.
- Budget Utilization (%) = (sum of the user's expenses in the budget's category between start_date and end_date ÷ budget_amount_kobo) × 100.
- The 80% warning threshold is stored as a configuration value in the code (for example WARNING_THRESHOLD = 0.80) so that it can be changed later without editing the calculation logic.

PHASE FIVE: USER INTERFACE DESIGN

5.1 Design Direction

The interface should feel like a real financial product and not a basic classroom demonstration. It uses a clean, modern fintech style:

- Professional dark navy or deep blue as the main brand colour.
- White and light neutral backgrounds for content areas.
- Green accents for income and positive balances.
- Red or orange accents for expenses and warnings.
- Clear typography, well-spaced cards, rounded corners and simple icons.
- Clean charts that are not crowded.

The design must work well on Android screens and on desktop screens. It will be prepared in Figma before it is coded.

5.2 Required Screens

No.	Screen	Main contents
1	Welcome / Landing page	Product name, short explanation, Register and Login buttons
2	Registration page	Full name, email, password, confirm password, validation messages
3	Login page	Email, password, error message, link to registration
4	Main dashboard	Total income, total expenses, available balance, active budget, budget utilization, recent transactions, expense categories, income-versus-expense chart, budget warnings
5	Add Income form	Amount, source, date, optional description
6	Add Expense form	Amount, category, date, optional description
7	Transaction History page	Combined list of income and expenses, search box, filters by category, date and type, edit and delete buttons

No.	Screen	Main contents
8	Budget Management page	List of budgets with progress bars, create budget button
9	Budget Details page	Budget limit, period, spending so far, utilization, related expenses
10	Financial Reports page	Date range selector, totals, spending by category, transactions in the period, budget performance, print option
11	User Profile page	View and update name and email, change password
12	Settings page	Application preferences, category management, logout

5.3 Dashboard Requirements

The dashboard displays real information from the database for the logged-in user, never hardcoded sample values. It shows total income, total expenses, available balance, the active budget, budget utilization, recent transactions, expense categories, an income-versus-expense chart and budget warning notifications. When a new user has not entered any transactions, the dashboard shows useful empty states, for example a message such as No transactions yet with a button to add the first income or expense.

5.4 Transaction Management Behaviour

- Users can add, edit and delete transactions.
- Users can search transactions and filter them by category, date and type.
- Income and expenses are clearly distinguished, for example green with a plus sign for income and red or orange with a minus sign for expenses.
- The application asks for confirmation before deleting a transaction.
- Amounts are always displayed in Nigerian Naira, for example ₦5,000.00.

5.5 Responsive Design and PWA Behaviour

On phones, the navigation appears as a bottom bar and content is shown in a single column. On tablets and computers, the navigation moves to a side bar and cards are arranged in a grid. The PWA includes a web app manifest, application icons and a service

worker so that compatible Android browsers can offer to add the application to the home screen. Because the application depends on the server for data, offline transaction entry is not supported in the initial version, and the interface should show a clear message when the internet connection is unavailable.

PHASE SIX: IMPLEMENTATION PLAN

This phase describes how the design will be turned into a working application. The application has not been built yet; this plan is the guide for the development stage.

6.1 Technology Stack

Area	Technology	Reason
Frontend	HTML5, CSS3, JavaScript, Bootstrap (optional)	Simple, widely taught and responsive
Backend	Python with the Flask framework	Small, readable framework that is easy to explain and defend
Database	SQLite	Needs no separate server, suitable for the initial version
Version control	Git and GitHub	Team collaboration and history of changes
UI/UX design	Figma	Screens are designed before coding
Development environment	VS Code	Free and popular editor
Deployment	To be decided (see Phase Ten)	Free hosting has limits that affect SQLite

No extra frameworks are introduced. If the group later needs a different database or an ORM, the design allows it, and the reason must be explained before the stack is changed.

6.2 Proposed Project Structure

```
budget_tracker/
  app/
    __init__.py        # creates the Flask app and loads configuration
    config.py          # settings, including WARNING_THRESHOLD
    db.py              # database connection (runs PRAGMA foreign_keys = ON)
    auth.py            # register, login, logout, login-required checks
    income.py          # income routes
    expenses.py        # expense routes
    budgets.py         # budget routes and monitoring
    reports.py         # report routes
    services/
      calculations.py  # balance, utilization and money conversion
      validation.py    # server-side input validation
    templates/         # HTML pages
    static/
      css/ js/ icons/
      manifest.json
      service-worker.js
  schema.sql           # database creation script (section 4.6.5)
  tests/               # unit, integration and security tests
  requirements.txt
  .env.example         # names of settings only, never real secrets
  .gitignore           # excludes .env, the database file and virtual
environment
```

6.3 Main Routes

Area	Routes	Purpose
Public	GET /	Welcome page
Authentication	GET and POST /register, GET and POST /login, POST /logout	FR01 to FR03
Profile	GET and POST /profile	FR04
Dashboard	GET /dashboard	FR13
Income	GET and POST /income/new, POST /income/id/edit, POST /income/id/delete	FR05, FR06

Area	Routes	Purpose
Expenses	GET and POST /expenses/new, POST /expenses/id/edit, POST /expenses/id/delete	FR07 to FR09
Transactions	GET /transactions with search and filter parameters	FR16, FR17
Budgets	GET /budgets, GET and POST /budgets/new, GET /budgets/id	FR10, FR11, FR15
Reports	GET /reports with start and end dates	FR14
Settings	GET /settings, GET and POST /categories	Categories and preferences

Every route except the welcome page, registration and login requires a logged-in user, and every record route checks that the record belongs to that user.

6.4 Money Handling and Calculation Logic

All amounts are stored as whole kobo to avoid decimal errors. The calculation functions are kept in one module so they can be unit tested.

```
from decimal import Decimal, InvalidOperation, ROUND_HALF_UP

WARNING_THRESHOLD = 0.80   # configurable: 80 percent

def naira_to_kobo(text):
    try:
        amount = Decimal(text).quantize(Decimal('0.01'),
rounding=ROUND_HALF_UP)
    except InvalidOperation:
        raise ValueError('Enter a valid amount')
    if amount <= 0:
        raise ValueError('Amount must be greater than zero')
    return int(amount * 100)

def kobo_to_naira(kobo):
    naira, rest = divmod(kobo, 100)
    return f'₦{naira:,}.{rest:02d}'

def available_balance(total_income_kobo, total_expenses_kobo):
    return total_income_kobo - total_expenses_kobo
```

```
def budget_utilization(expenses_kobo, limit_kobo):
    return expenses_kobo / limit_kobo * 100

def budget_status(utilization):
    if utilization > 100:
        return 'over_budget'
    if utilization >= WARNING_THRESHOLD * 100:
        return 'warning'
    return 'normal'
```

Summaries are not stored; they are recalculated from the database each time they are displayed, so adding, editing or deleting a transaction automatically updates the dashboard and reports (business rule BR07).

6.5 Budget Periods

A daily budget covers one day, a weekly budget covers seven days starting on the chosen start date, and a monthly budget covers the period from the start date to the day before the same date in the next month. The system calculates the end date from the start date and the period. When the budget utilization is calculated, only expenses in the budget's category whose expense_date falls between start_date and end_date are counted.

6.6 Notifications

The initial version provides in-app notifications only: a warning banner at 80% utilization and an over-budget alert above 100%. The budget_alerts table prevents the same alert status from being shown repeatedly. Push notifications are not part of the initial version and must not be claimed unless they are implemented and tested.

6.7 Progressive Web Application Features

- A web app manifest with the application name, theme colour and icons (for example 192 by 192 and 512 by 512 pixels).
- A service worker that caches static files such as stylesheets, scripts and icons.
- HTTPS in the hosted version, which browsers require before they offer installation.
- Responsive layout for phones, tablets and computers.

Honest offline limitation: the service worker only speeds up loading of static files and shows an offline message. Recording transactions without an internet connection and synchronising them later is not implemented in the initial version, so the system must not

be described as supporting offline use.

6.8 Development Order

1. Set up the repository, the Flask project and the database script.
2. Build registration, login, logout and protected pages.
3. Build categories and income management.
4. Build expense management.
5. Build budgets, calculations and notifications.
6. Build the dashboard and transaction history with search and filters.
7. Build financial reports.
8. Add PWA files and polish the responsive interface.
9. Complete testing and fix defects.
10. Deploy and prepare the final presentation.

PHASE SEVEN: SECURITY DESIGN

The application stores personal financial information, so security is a core requirement (NFR03).

No.	Measure	How it will be applied
1	Password hashing	Passwords are hashed with a recognised function such as the ones provided by Werkzeug; plain-text passwords are never stored
2	Authentication checks	Private pages require a valid session; otherwise the user is redirected to the login page
3	Authorization checks	A record is only returned or changed when its user_id matches the logged-in user
4	Server-side validation	The backend validates amounts, dates, categories, periods and text lengths even if the frontend already checked them
5	Safe database queries	All queries use parameters (placeholders) and never join user input into SQL text, which protects against SQL injection

No.	Measure	How it will be applied
6	CSRF protection	Every form that changes data includes a CSRF token that the backend verifies
7	Secure session configuration	Secure, HttpOnly and SameSite cookie settings and a sensible session lifetime
8	Secret management	The secret key and other sensitive settings are read from environment variables; the .env file is listed in .gitignore
9	User-specific queries	Every query on financial tables includes WHERE user_id = the logged-in user
10	Safe error messages	Login errors do not reveal whether an email exists, and error pages do not display internal details

Secret keys and passwords must never be committed to GitHub.

PHASE EIGHT: TESTING PLAN

8.1 Testing Levels

Level	Purpose	Examples
Unit testing	Test individual functions	Balance calculation, budget utilization, money conversion, input validation
Integration testing	Test communication between parts	Frontend and backend, backend and database, authentication and protected pages
System testing	Test the whole application from the user's point of view	Register, add transactions, create budgets, view dashboard and reports
Usability testing	Check whether a student can use the application without difficulty	Ask a few students to add an income, an expense and a budget without help and note where they struggle

Level	Purpose	Examples
Security testing	Check protection of data	Try to open another user's records, submit invalid requests, submit forms without a CSRF token

8.2 Test Cases

The tests below have been designed but **not yet executed**, because the application has not been built. The actual result and pass/fail columns must be filled in only after the tests are run.

Test ID	Description	Test data	Expected result	Actual result	Status
TC01	Register with valid details	Name Ada Obi, email ada@example.com, strong password	Account created and user can log in	Not yet executed	Not run
TC02	Register with an existing email	Same email as TC01	Registration rejected with a message that the email is already registered	Not yet executed	Not run
TC03	Login with correct credentials	ada@example.com and the correct password	Session created and dashboard displayed	Not yet executed	Not run
TC04	Login with incorrect credentials	ada@example.com and a wrong password	Login rejected with a generic error message	Not yet executed	Not run
TC05	Add valid income	₦50,000.00, source Allowance, valid date	Income saved and total income and balance increase	Not yet executed	Not run
TC06	Add valid expense	₦2,500.00, category Feeding, valid date	Expense saved and total expenses increase and balance decreases	Not yet executed	Not run
TC07	Reject zero or negative amounts	Amounts 0 and -500 for income and expense	Both rejected with an error; nothing saved	Not yet executed	Not run
TC08	Edit an expense	Change the ₦2,500.00 expense to ₦3,000.00	Record updated and dashboard totals recalculated	Not yet executed	Not run
TC09	Delete an expense	Delete the expense after confirming	Confirmation shown, record removed and totals recalculated	Not yet executed	Not run

Test ID	Description	Test data	Expected result	Actual result	Status
TC10	Create a budget	Feeding, ₦10,000.00, monthly, valid start date	Budget saved with the correct end date	Not yet executed	Not run
TC11	Display a warning at 80%	Feeding expenses totalling ₦8,000.00 against the ₦10,000.00 budget	Utilization 80% and a warning is displayed	Not yet executed	Not run
TC12	Display an alert when a budget is exceeded	Add ₦3,000.00 more Feeding expenses (total ₦11,000.00)	Utilization 110% and an over-budget alert is displayed	Not yet executed	Not run
TC13	Calculate the available balance correctly	Total income ₦50,000.00 and total expenses ₦12,500.00	Available balance is ₦37,500.00	Not yet executed	Not run
TC14	Filter transactions by date	Records dated 1 and 5 of a month; filter 1 to 3	Only the record dated the 1st is listed	Not yet executed	Not run
TC15	Generate a financial report	Select a period that contains known transactions	Totals, spending by category and budget performance match the records	Not yet executed	Not run
TC16	Prevent access to another user's records	User A requests the edit or delete address of User B's expense	Request rejected (not found or forbidden); no data shown or changed	Not yet executed	Not run

8.3 Recording Results

For each executed test, the tester records the date, the actual result, the pass or fail status and, for failures, a defect description. Failed tests are fixed and re-tested before the final presentation.

PHASE NINE: PROJECT MANAGEMENT AND TEAM STRUCTURE

9.1 Team Responsibilities

The group has 10 members. A practical division of responsibilities is proposed below. Members can support each other, but each role has one person who is accountable.

No.	Role	Main responsibilities
1	Project Manager and Team Coordinator	Plans the schedule, holds meetings, tracks progress and manages risks
2	Requirements Analyst	Maintains the requirements and business rules and validates them with students
3	System and UML Designer	Produces the use case, activity, sequence and architecture diagrams
4	Database Architect	Designs and maintains the database and the creation script
5	UI/UX Designer	Designs the screens in Figma and keeps the interface consistent
6	Frontend Developer	Builds pages, forms, charts and the responsive layout
7	Backend Developer	Builds Flask routes, validation and business logic
8	Security and Authentication Developer	Builds login, sessions, CSRF protection and authorization checks, and reviews security
9	Software Tester	Writes and runs the test cases and reports defects
10	Documentation and Presentation Lead	Maintains the project documentation and prepares the PowerPoint presentation

9.2 Version Control Workflow

The team uses one GitHub repository. The main branch always holds working code. Each member creates a branch for a feature, commits small changes with clear messages and opens a pull request that another member reviews before merging.

PHASE TEN: DEPLOYMENT AND MAINTENANCE

10.1 Hosting Consideration

Render is a practical option for hosting a Flask application, but its limits affect this project. According to Render's documentation, a free web service spins down after 15 minutes without traffic, and free web services have an ephemeral filesystem, which means local SQLite databases are lost whenever the service redeploys, restarts or spins down. Persistent disks are available only on paid instance types, while relational data on the free tier can instead be kept in a Render Postgres database. These details can change, so the group must check Render's current documentation before deploying.

10.2 Options for the Group

Option	Description	Consequence
A. Demonstration only	Deploy with SQLite on a free instance	Easy, but all registered users and transactions can disappear, and the first request after idle can be slow; acceptable only for a short demonstration
B. Paid instance with persistent disk	Keep SQLite on an attached disk	Data persists, but it has a cost
C. Managed PostgreSQL	Use SQLAlchemy and switch the production database to Postgres while using SQLite locally	Data persists, but requires a small design change; the reason for the change would be explained before it is made

For a university defence, the group should run the application locally or use Option A with a clear statement of its limitation, and use Option B or C if the application must keep real data.

10.3 Maintenance

After deployment, the team will fix defects, back up the database, update dependencies and review security settings. Future improvements could include push notifications, offline entry with synchronisation, data export, and a native Android application if required.

PHASE ELEVEN: CONCLUSION

This document has presented the introduction, requirements analysis, feasibility study, system design, user interface design, implementation plan, security design, testing plan, team structure and deployment considerations of the Student Budget and Expense Tracker. The project is feasible within the available resources when its scope is controlled. The documented requirements, diagrams and database design provide a clear foundation for building the application in the next stage.